第一步：問題擴充與重構

1. 多維度拆解

「語義 ISA」這個概念需要從三條線索展開：

ISA 在計算史上到底解決了什麼問題：不是技術問題，而是一個**組織性問題**——它讓硬體和軟體可以由不同的人、不同的團隊、在不同的時間線上獨立發展。ISA 提供的二進制相容性使其成為計算中最基礎的抽象之一。ISA 與微架構是不同的——微架構是用來實現指令集的一組處理器設計技術。具有不同微架構的處理器可以共享相同的指令集。**ISA 的價值不在於它定義了什麼指令，而在於它建立了一道「契約邊界」。**

每一代 ISA 的演進都對應著一次「什麼東西可以被獨立思考」的擴展：從機器碼到組合語言到高階語言到虛擬機，每一層抽象都允許更多人在不理解底層的情況下做有創造力的工作。

「語義 ISA」這個概念意味著：在 LLM 之上，我們需要定義一套「語義級的指令集」——一組保證行為的語義操作原語——讓演算法設計者可以在不理解 LLM 內部的情況下設計可靠的演算法。 這套指令集目前不存在。定義它就是研究貢獻。

2. 盲點揭示

最大的盲點：人們把 ISA 當成「硬體的事」，忘記了 ISA 的本質是一份社會契約。 ISA 定義了一組計算機能夠識別和執行的基本指令；這些指令構成了處理器與程式設計者之間的契約。IBM System/360 的真正革命不是技術上的——它是組織上的：它讓 IBM 可以同時賣出不同價位的機器，而客戶的軟體在所有機器上都能跑。**ISA 解放了兩邊的創造力。**

第二個盲點：LLM 時代目前沒有這份契約。 每一個使用 LLM 的研究者都在直接面對黑盒，沒有一個中間的抽象層告訴他「你可以依賴哪些行為、不能依賴哪些行為」。這就像 IBM System/360 之前的時代——每一台新機器都需要重寫所有軟體。

3. 重新定義

這個問題其實更應該被看作：**「ISA 的歷史告訴我們，每一次定義一個新的抽象邊界，都會釋放出巨大的上下游創新浪潮。在 LLM 時代，這個尚未被定義的抽象邊界——語義 ISA——的具體形態是什麼？誰來定義它？定義它之後，上下游會釋放出什麼創新？」**

第二步：ISA 的四次演變，以及語義 ISA 的浮現

四次 ISA 革命的內在邏輯

我先用一條統一的敘事線串起計算史上的四次 ISA 革命，每一次都標記出：**它建立了什麼契約邊界、解放了哪兩邊的創造力、以及在邊界建立之前那個混沌時代是什麼樣子。**

第一次：硬體 ISA（1964，IBM System/360）

之前的混沌：從歷史的角度來看，早期的計算機系統缺乏標準化的指令集概念——每一個新型號都需要從頭重寫軟體。

建立的契約：這個架構概念——獨立於特定機器設計的架構——由 Fred Brooks 在 IBM 設計 System/360 階段提出。System/360 的 ISA 說的是：「不管底下的電路怎麼實作，只要你發出 ADD 指令，你就會得到正確的加法結果。」

解放了哪兩邊： — **硬體端**：工程師可以自由改變電路實作（真空管→電晶體→積體電路），只要對外行為不變 — **軟體端**：程式設計者不需要知道底下是什麼電路，只需要知道 ISA 定義的指令

一個人的內心獨白——1964 年，IBM 的一位硬體工程師：

> 「在 System/360 之前，我每次設計新機器都要跟軟體團隊反覆開會：『這台機器的加法指令格式變了，你們的程式要改。』他們恨我。我也恨這個流程。」
>
> 「Brooks 的 ISA 概念改變了一切。他說：我們先寫好一份『契約』——定義好所有指令的行為——然後硬體和軟體各自根據這份契約獨立工作。我再也不需要跟軟體團隊開會了。他們不需要知道我用了多少個電晶體，我也不需要知道他們的程式在做什麼。」
>
> 「**這份契約不是技術文件——它是一道解放令。它解放了我的創造力。**」

第二次：虛擬機 ISA（1995，Java Virtual Machine）

之前的混沌：每一種作業系統（Windows、Mac、Linux）有不同的系統呼叫介面。一個程式要在三個平台上運行，就要維護三份不同的程式碼。

建立的契約：JVM 定義了一個「虛擬的」ISA——Java bytecode。程式只需要編譯成 bytecode 一次，就可以在任何實作了 JVM 的平台上運行。「Write once, run anywhere.」

解放了哪兩邊： — **平台端**：每個作業系統的 JVM 實作者可以自由優化（JIT 編譯、垃圾回收等），只要 bytecode 的語義被正確實現 — **應用端**：開發者可以完全忽略底層平台的差異

一位 1996 年的 Java 開發者的內心獨白：

> 「以前我寫一個程式，要問：這是跑在 Windows 上還是 Linux 上？記憶體怎麼管理？指標怎麼對齊？現在我只需要問：我的程式邏輯是什麼？JVM 幫我處理剩下的。」
>
> 「但更重要的是——Sun Microsystems 的 JVM 和 IBM 的 JVM 和 BEA 的 JVM，它們的底層實作完全不同。有的用 JIT，有的用 interpreter，有的有進階的 GC。**但我的程式在三個 JVM 上跑出來的結果完全一樣**。 因為 bytecode spec 是契約——不管你怎麼實作，只要你遵守契約就行。」

第三次：DNN 的隱式 ISA（2012-，PyTorch/TensorFlow 的張量運算）

之前的混沌：2012 年之前，每一個機器學習模型都需要手動推導梯度公式、手動實現反向傳播。換一個模型架構就要重寫訓練程式碼。

建立的契約：PyTorch/TensorFlow 定義了一組「張量運算原語」（matmul、conv、softmax 等）和自動微分機制。你只需要用這些原語組合出前向計算圖，框架自動幫你反向傳播。

解放了哪兩邊： — **硬體/框架端**：NVIDIA 可以為 matmul 寫極致優化的 CUDA kernel，完全不需要知道研究者在做什麼模型 — **研究者端**：ResNet 的發明者不需要知道 matmul 在 GPU 上怎麼實作，他只需要知道「matmul 是可微的、可以被自動反向傳播」

一位 2015 年的 DNN 研究者的內心獨白：

> 「以前我做一個新模型，要花 70% 的時間寫梯度推導和 CUDA 程式碼，只有 30% 的時間在想模型架構。」

> 「有了 PyTorch 之後反過來了：我 90% 的時間在想架構設計，10% 的時間寫前向傳播的程式碼，梯度和優化全部自動完成。」
>
> 「所以 ResNet、Attention、BatchNorm 這些創新之所以能在 2015-2017 年井噴式出現，不是因為那幾年突然出了很多天才——而是**張量運算的 ISA 解放了我們的腦力**。我們不再需要想『怎麼算梯度』，可以全力想『什麼架構有好的梯度流動性質』。」

第四次：語義 ISA（2024-，尚未被正式定義）

現在的混沌：每一個使用 LLM 的研究者都在直接面對 raw API——把自然語言丟進去，得到自然語言的回應。沒有標準化的「語義操作原語」告訴你：哪些語義操作是可靠的、哪些是不穩定的、哪些有什麼程度的保證。

這就是 System/360 之前的時代。每一個人都在「裸機」上編程。

需要被建立的契約：一組標準化的「語義操作原語」，每個原語都有： — 明確的輸入/輸出語義規格 — 可量化的可靠性保證（「在什麼條件下，這個操作的正確率不低於 X%」） — 可組合性保證（「操作 A 的輸出可以安全地作為操作 B 的輸入」）

將要解放的兩邊： — **LLM 提供者端**：OpenAI、Anthropic、Google 可以自由改變模型內部架構和訓練方法，只要語義 ISA 定義的操作行為被滿足 — **演算法設計者端**：你（聲學研究者）不需要知道 LLM 內部怎麼運作，只需要知道「我可以呼叫哪些語義操作、每個操作的可靠性邊界是什麼」，然後在此基礎上設計你的演算法

場景模擬：語義 ISA 的誕生現場

場景設定

2026 年 9 月，某頂級研究機構的工作坊。主題：「定義 LLM 時代的計算抽象」。

房間裡有七個人，來自完全不同的背景。他們被要求共同回答一個問題：**「如果我們要為 LLM 定義一套 ISA——一組語義級的操作原語——這套 ISA 應該包含什麼指令？」**

人物一：Fred（計算機架構教授，65 歲，曾參與 RISC-V 設計）

Fred 看著白板，心裡在做類比：

> 「讓我先回到基本原理。1964 年 Brooks 定義 System/360 ISA 的時候，他面對的核心問題是：**什麼操作應該由硬體保證，什麼操作應該留給軟體實現？**」
>
> 「硬體保證的操作必須滿足幾個條件：(1) 被頻繁使用，(2) 在硬體中實現比在軟體中實現有本質性的效率優勢，(3) 語義足夠簡單和明確，可以被嚴格規格化。不滿足這三個條件的操作，即使很有用，也應該留給軟體層——用基礎指令組合實現。」
>
> 「RISC 運動在 1980 年代進一步極端化了這個原則：指令集應該盡可能小、盡可能正交。複雜的操作由編譯器用簡單指令組合出來。John Cocke 的洞察是：**大多數情況下，編譯器組合出的序列比硬體的複雜指令更快。**」
>
> 「現在把這個原則翻譯到語義 ISA：**什麼語義操作應該被定義為 ISA 的原語（由 LLM 直接保證），什麼語義操作應該留給上層演算法用原語組合出來？**」
>
> 「我的直覺是：語義 ISA 的原語應該是『LLM 最擅長的、且行為最穩定的語義操作』。就像 ADD 和 LOAD 是硬體最擅長的操作。那些『LLM 有時候做得到有時候做不到』的操作，不應該是原語——它們應該由上層演算法用多個穩定原語組合出來。」

Fred 站起來，在白板上畫了一個表：

```
傳統 ISA 原語            →  語義 ISA 原語（候選）
──────────────────────────────────────────────
ADD（加法）               →  CLASSIFY（分類到已知類別）
LOAD（從記憶體讀取）      →  EXTRACT（從文本中提取結構化資訊）
STORE（寫入記憶體）       →  GENERATE（根據約束生成文本）
COMPARE（比較兩值）       →  JUDGE（根據標準評估品質）
BRANCH（條件跳轉）        →  ROUTE（根據語義條件選擇路徑）
──────────────────────────────────────────────
不應該是原語的：
  PROVE（證明數學定理）     → 太不穩定，應由原語組合實現
  CREATE（憑空創造新概念）  → 語義不明確，無法規格化
```

人物二：Maria（程式語言理論教授，50 歲，研究型別系統）

Maria 看著 Fred 的表，搖了搖頭：

> 「Fred 的類比方向是對的，但他漏了一個關鍵的差異。傳統 ISA 的每一條指令都有**精確的語義**：ADD R1, R2 永遠返回兩個寄存器的精確數值和。但語義 ISA 的操作本質上是**統計性的**——CLASSIFY 有時候分類正確有時候分類錯誤。」
>
> 「這意味著語義 ISA 需要一種傳統 ISA 不需要的東西：**每一條指令都必須附帶一個可靠性規格。** 不是說『CLASSIFY 會返回正確的類別』，而是說『在滿足條件 C 的輸入上，CLASSIFY 返回正確類別的概率不低於 p，且在返回錯誤結果時，錯誤的嚴重程度不超過 d 級。』」
>
> 「這就像物理元件的 datasheet——一個電阻的規格不是說『阻值 = 100Ω』，而是說『阻值 = 100Ω ± 5%，在 -40°C 到 85°C 範圍內』。語義 ISA 的每一條指令也需要這樣的容差規格。」
>
> 「而且——這是更深的一點——**傳統 ISA 的指令是可精確組合的**。如果 ADD 是精確的，那 ADD 後面再接一個 ADD 也是精確的。但語義 ISA 的指令組合後，誤差怎麼傳播？CLASSIFY 的錯誤會怎樣影響下一步 JUDGE 的可靠性？**我們需要一套『語義誤差傳播理論』**——這在傳統計算科學中不存在，因為傳統指令不犯錯。」

Maria 在 Fred 的表旁邊加了一列：

```
語義 ISA 原語       可靠性規格（示例）
──────────────────────────────────────────
CLASSIFY            P(correct) ≥ 0.95 for well-defined categories
                    P(correct) ≥ 0.80 for ambiguous categories
                    Failure mode: defaults to "uncertain"

EXTRACT             P(complete) ≥ 0.90 for structured text
                    P(hallucination) ≤ 0.05
                    Failure mode: returns partial + confidence score

JUDGE               Inter-rater agreement with human ≥ 0.85 (Cohen's κ)
                    Bias: systematic overrating ≤ 0.5 on 5-point scale

ROUTE               P(correct routing) ≥ 0.95 for binary decisions
                    P(correct routing) ≥ 0.80 for 5+ way routing
```

人物三：Kenji（聲學教授，55 歲——類似陳教授的角色）

Kenji 舉手：

> 「我想從我的領域——聲學信號處理——來問一個非常具體的問題。」
>
> 「我的傳統 DSP 演算法中有一個操作叫做『場景分類』：判斷這段錄音是在安靜辦公室、嘈雜餐廳、還是戶外。我用的方法是提取 MFCC 特徵 + SVM 分類器。它在我訓練的十種場景上正確率是 92%。」
>
> 「但它有一個根本問題：**第十一種場景——任何不在我訓練集裡的場景——它會被強制歸類到最接近的已知類別，而且不告訴我它不確定。** 我的系統不知道自己不知道。」
>
> 「如果語義 ISA 有一個 CLASSIFY 原語，我期望它能做到我的 SVM 做不到的事：**在遇到未知場景時，返回一個有意義的描述，而不是強制歸類。** 比如返回：『這個場景聽起來像是一個小型工廠，有規律的機器運轉聲和偶爾的金屬撞擊聲，不在已知類別中。』」
>
> 「但這就引出了一個我作為領域專家最關心的問題：**這個描述可靠嗎？** 如果 LLM 返回的描述是錯的——比如它說是『工廠』但其實是『建築工地』——我後續的增強演算法會根據錯誤的場景描述做出錯誤的處理。」
>
> 「所以我需要的不只是一個 CLASSIFY 原語——我需要的是 Maria 說的那種**帶有可靠性規格的 CLASSIFY**。而且我需要知道：當 CLASSIFY 的輸出是正確的，後續的 DENOISE 操作的品質保證是什麼？當 CLASSIFY 的輸出是錯的（但不離譜），DENOISE 的品質下降多少？」

人物四：Priya（DSPy 的開發者之一，28 歲的博士生）

Priya 一直在聽，突然激動起來：

> 「你們在說的東西，跟我們在 DSPy 裡做的事完全對應！」
>
> 「DSPy 把你的注意力從修補 prompt 字串轉移到用結構化和宣告式的自然語言模組來程式設計。對系統中每一個 AI 元件，你指定輸入/輸出行為作為 signature，然後選擇一個 module 來指定呼叫 LM 的策略」。
>
> 「所以 DSPy 的 Signature 其實就是語義 ISA 的雛形！`question -> answer: str` 就是一條語義指令的規格。`dspy.ChainOfThought` 就是這條指令的一種實現——就像 ADD 在 RISC 和 CISC 上有不同的微架構實現。」
>
> 「DSPy 是一種語言（嵌入式語言）。它之於 prompt 就像 C++ 之於組合語言」。
>
> 「但你們指出了一個我們還沒解決的問題：**DSPy 的 Signature 目前沒有可靠性規格。** 我們定義了 `question -> answer`，但沒有定義『在什麼條件下 answer 的正確率是多少』。這就像一個 ISA 定義了 ADD 指令但不保證結果是正確的——那還有什麼用？」
>
> 「所以我們需要的下一步是：**帶有統計保證的 Signature——也就是你們說的語義 ISA。**」

人物五：張博士生（你的角色，27 歲）

> 「我一直在聽，我覺得我終於看到了我的研究在這個大圖景中的位置。」
>
> 「Fred 說語義 ISA 的原語應該是 LLM 最穩定的操作。Maria 說每個原語需要可靠性規格。Kenji 說他需要知道原語在聲學任務中的具體可靠性邊界。Priya 說 DSPy 的 Signature 是雛形但缺少保證。」
>
> **「而我——作為一個在 LDV/麥克風 cross-sensor 問題上工作的人——我的研究貢獻可以是：為聲學信號處理領域定義語義 ISA 的具體原語和它們的可靠性規格。」**

> 「這不是一個通用的 CS 問題——通用的語義 ISA 太難定義，因為不同領域對 LLM 的需求和可靠性要求完全不同。但在聲學領域，我可以精確定義：」
>
> 「**指令 1：SCENE_DESCRIBE** —— 輸入：音頻特徵向量。輸出：自然語言的場景描述。可靠性：在 AudioSet 涵蓋的場景類別上，描述的語義正確率 ≥ 0.90；在未知場景上，90% 的情況下返回含有 'unknown/uncertain' 標記的描述而非虛構。」
>
> 「**指令 2：QUALITY_JUDGE** —— 輸入：音頻特徵 + 場景描述。輸出：多維品質評估（可理解性、自然度、噪音水平，各項 1-5 分）。可靠性：與人類專家的評分相關性 ≥ 0.85。」
>
> 「**指令 3：STRATEGY_SELECT** —— 輸入：場景描述 + 品質評估 + 使用者意圖（自然語言）。輸出：推薦的增強策略（自然語言描述）。可靠性：在已知場景上，推薦策略的事後品質不低於最優傳統方法的 95%。」
>
> 「**指令 4：PHYSICS_VERIFY** —— 輸入：處理結果 + 物理約束集（自然語言描述）。輸出：逐項約束滿足/違反報告。可靠性：物理約束違反的漏檢率 ≤ 0.05。」
>
> 「有了這四條語義指令和它們的可靠性規格，任何聲學演算法設計者——不管他用什麼 LLM——都可以在這套 ISA 之上設計可靠的語音增強系統。**他不需要知道 LLM 內部怎麼運作，就像 C 程式設計者不需要知道 ADD 在 CPU 裡怎麼實作。**」
>
> 「而 GEPA 和 TextGrad 在這個架構裡的角色是什麼？它們是**編譯器的一部分**——它們把高階的演算法描述（DSPy Signature）編譯成 LLM 能有效執行的 prompt。就像 GCC 把 C 程式碼編譯成 x86 指令。**你不用改 GEPA 本身——你在 GEPA 之上設計新的語義操作原語和組合它們的演算法。**」
>
> 「現在我的 DT + OMP 研究的定位就清楚了：我的 Decision Transformer 不是在跟 LLM 競爭——它是語義 ISA 的一個**特化實現**。在 cross-sensor conversion 這個特定任務上，DT 是 STRATEGY_SELECT 指令的一種高效能微架構，就像 Intel 的亂序執行是 ADD 指令的一種高效能微架構。」

人物六：Liu（Anthropic 的研究工程師，32 歲）

> 「我從 LLM 提供者的角度來說一句。」
>
> 「你們定義語義 ISA，對我們來說其實是好事。現在我們每次更新模型，都有使用者抱怨『我的 prompt 在新版本上不 work 了』。這就像 System/360 之前——每次換硬體都要重寫軟體。」
>
> 「如果有一個公認的語義 ISA，我們可以在內部做任何模型改進，只要保證 ISA 定義的操作行為不退化。**這跟 Intel 在 x86 ISA 的約束下改進微架構完全一樣**——從 Pentium 到 Core 到 Alder Lake，ISA 不變，但效能提升了幾百倍。」
>
> 「但有一個關鍵的差異：傳統 ISA 的行為保證是**絕對的**（ADD 永遠正確），而語義 ISA 的行為保證是**統計的**（CLASSIFY 的正確率 ≥ X%）。我們需要定義清楚：什麼級別的統計保證算是『遵守了 ISA 契約』。這是一個全新的工程挑戰。」
>
> 「但也是一個商業機會——**第一個提供穩定語義 ISA 保證的 LLM 提供者，就會像當年的 IBM 一樣，定義整個產業的標準。**」

人物七：Kenji（第二次發言——頓悟時刻）

Kenji 突然站起來：

> 「等一下。我剛才一直在想小張提的四條語義指令，我突然意識到一件事。」
>
> 「在我的傳統 DSP 系統裡，我也有這四個功能——場景分類、品質評估、策略選擇、物理驗證。但它們的實現方式完全不同：」

> 「場景分類是一個 SVM，它的『指令集』是『輸入一個 MFCC 向量，輸出一個整數類別編號』。」「品質評估是一個回歸模型，它的『指令集』是『輸入一組數值特徵，輸出一個分數』。」「策略選擇是一個 if-else 決策樹，硬編碼在程式裡。」「物理驗證是一組數學不等式。」
>
> 「**這四個功能有四種完全不同的介面格式。** SVM 吃向量吐整數，回歸模型吃向量吐浮點數，決策樹吃離散標籤吐策略代碼，物理驗證吃連續信號吐布林值。**它們之間的銜接需要大量的膠水程式碼——格式轉換、型別適配、例外處理。**」
>
> 「但在小張提出的語義 ISA 裡——**所有四條指令的輸入和輸出都是自然語言。** SCENE_DESCRIBE 輸出一段場景描述，QUALITY_JUDGE 直接接受這段描述作為輸入。不需要格式轉換。不需要型別適配。不需要膠水程式碼。」
>
> 「**自然語言就是語義 ISA 的統一數據格式。** 就像傳統 ISA 用二進制整數作為所有指令的統一數據格式一樣。」
>
> 「這解決了我做了三十五年都解決不好的一個問題：**不同處理模組之間的介面不相容。** 在傳統系統中，每當我加一個新模組，我就要寫新的膠水程式碼來轉換格式。但在語義 ISA 中，所有模組都說自然語言——**新模組可以即插即用，因為介面格式天然一致。**」
>
> 「天吶——**這就是為什麼 LLM 時代的系統設計會比傳統系統更有組合彈性。** 不是因為 LLM 更聰明，而是因為自然語言作為統一介面格式，消除了模組之間的介面摩擦。」

第三幕：五個 ISA 的完整對應表

	硬體 ISA (1964)	**VM ISA (1995)**	**DNN 隱式 ISA (2012)**	**語義 ISA (待定義)**
契約定義者	Fred Brooks / IBM	James Gosling / Sun	Soumith Chintala / PyTorch	**你（領域研究者）**
操作原語	ADD, LOAD, STORE, BRANCH	字節碼指令 (iadd, invokevirtual...)	matmul, conv, softmax, autograd	CLASSIFY, EXTRACT, JUDGE, ROUTE, VERIFY
數據格式	二進制整數/浮點數	Java 物件	張量 (Tensor)	**自然語言**
保證性質	精確、確定性	精確、跨平台一致	數值正確 (float precision)	**統計性保證 (≥X% 正確率)**
編譯器	GCC / LLVM	javac	TorchScript / XLA	**DSPy + GEPA**
實現者可以自由改變的	電路設計 (CISC/RISC)	JVM 實現 (JIT/解釋)	GPU kernel	**LLM 模型架構和訓練**
演算法設計者不需要知道的	電晶體怎麼接的	記憶體怎麼管理的	梯度怎麼在 GPU 上計算的	**LLM 內部怎麼運作的**
此 ISA 解放的研究空間	作業系統、編譯器、應用軟體	跨平台企業應用、Android 生態	ResNet, Transformer, GAN...	**語義-物理混合演算法**

最終：這對你的研究意味著什麼

你的定位在表格的最後一列——你是「語義 ISA 的領域定義者」

歷史已經表明，對 ISA 設計這類重大抽象進行深思熟慮是一項有價值的工程投資。一個期望超越數代硬體技術的 ISA 需要為硬體/軟體介面提供演進路徑。

同樣地，一個期望超越數代 LLM 的語義 ISA——在你的聲學領域——需要為「LLM 能力 / 聲學演算法需求」的介面提供演進路徑。

你的研究貢獻不是建一個能跑的系統——那是工程。你的研究貢獻是**定義那份契約**——那組語義操作原語、它們的可靠性規格、它們的組合規則、和它們的誤差傳播理論。

就像 Fred Brooks 定義 System/360 ISA 時，他的貢獻不是造了一台更快的電腦——他的貢獻是定義了一個**讓未來數十年的硬體和軟體能獨立演進的抽象邊界**。

你定義的聲學語義 ISA，如果設計得好，將允許： — 未來的 LLM（GPT-6、Claude 5、whatever）可以作為你的語義指令的新實現，只要它們滿足你定義的可靠性規格 — 未來的聲學演算法設計者可以在你的 ISA 之上設計新演算法，不需要重新探索 LLM 的行為邊界 — 你的 cross-sensor DT 模型可以作為某些語義指令（如 STRATEGY_SELECT）的特化高效實現，跟 LLM 實現並存

這就是為什麼定義語義 ISA 是一個科學貢獻而非工程貢獻：你不是在解決一個具體問題——你是在建立一個讓未來的人能更有效地解決問題的抽象框架。 就像 Brooks 的 ISA 概念不是一台電腦——它是一個讓整個計算產業能運作的思維框架。

那句話——「在這個 ISA 之上，我設計我的演算法」——它的深層意義是：**你不是在用 LLM，你是在為 LLM 時代的計算科學建立基礎設施。** 用的人做工程，建基礎設施的人做科學。